Report of DQN for Navigation Problem (Udacity DRLND)

Jun-Jie Zhang@IDRL

1. Implementation and model architectures

We mainly follow the algorithm of deep Q-learning with experience replay:

Initialize replay memory with capacity **BUFFER_SIZE**.

Set up action-value function Q (neural net) with random weights θ.

Set up target action-value function $\hat{Q}$ with weights $\theta^- = \theta$.

For episode $= 1,\ M$ **do**

Initialize environment and acquire initial state s_1.

For $t = 1, T$ **do**

Select action a_t using epsilon-greedy, with varying $eps = max(eps_end, eps * eps_decay)$ and $Q(\theta)$.

Execute action a_t in environment and observe reward r_t, state s_{t+1} and done information.

Store tuple $(a_t, s_t, r_t, s_{t+1}, done)$ in replay memory.

If replay memory is full,

Randomly select **BATCH_SIZE** minibatches of tuples $(a_j, s_j, r_j, s_{j+1}, done)$.

If $(t+1)\%$**UPDATE_EVERY** $== 0$,

Set $y_j = \begin{cases} r_j & \text{if done} = 1 \\ r_j + GAMMA \times max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$.

Train the net $Q(\theta)$ on $(y_j - Q(\theta))^2$, and update $\theta^- = \theta$.

End **for**

End **for**

Architecture of the Q net.

We use four-layer fully connected nets as a surrogate of the Q-table. Each layer (except for the last layer) is applied a leaky relu activation function. This net is sketched as follows:

```
self.fc1 = nn.Linear(state_size, 256)
self.fc2 = nn.Linear(256, 128)
self.fc3 = nn.Linear(128, 64)
self.fc4 = nn.Linear(64, action_size)

layer = F.leaky_relu(self.fc1(state))
layer = F.leaky_relu(self.fc2(layer))
layer = F.leaky_relu(self.fc3(layer))
layer = self.fc4(layer)
```

Figure 1 neural net layers

2. Parameter set up and preliminary results

Table 1. shows the values for the parameters above.

Param.	value	Param.	value
BUFFER_SIZE	10^5	LR	5×10^{-4}
BATCH_SIZE	64	UPDATE_EVERY	4
GAMMA	0.99	TAU	5×10^{-3}
eps_start	1.0	eps_decay	0.995
eps_end	0.		

Table 1 Parameters

Figure 2. plots the rewards per episode.

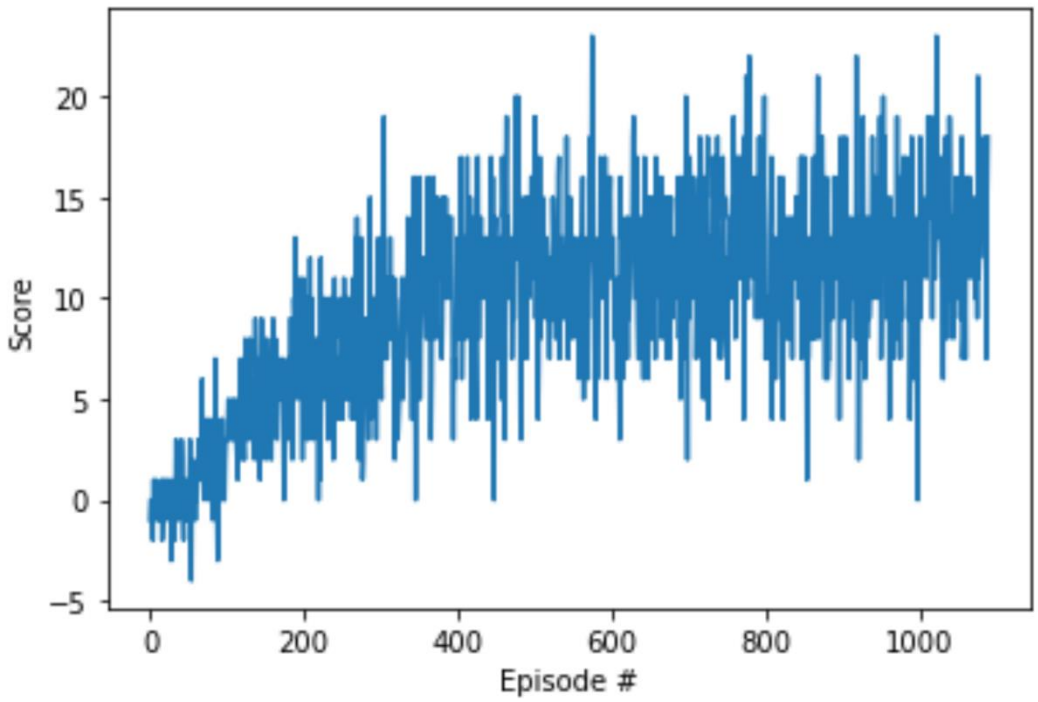

Figure 2 rewards per episode

Table 2. lists the average reward per 100 episodes, and the agent is able to receive an average reward (over 100 episodes) of 13.02 in 989 episodes.

Episode 100	Average Score: 0.71
Episode 200	Average Score: 5.15
Episode 300	Average Score: 7.18
Episode 400	Average Score: 9.78
Episode 500	Average Score: 11.37
Episode 600	Average Score: 11.49
Episode 700	Average Score: 11.25
Episode 800	Average Score: 12.81
Episode 900	Average Score: 11.64
Episode 1000	Average Score: 11.98

Episode 1089	Average Score: 13.02

Table 2 average rewards per 100 episodes

3. Tendencies that might improve the performance

Observation1. In the training, we find that the four-layer (nodes: 256,128,64,4) model performs a bit better than the three-layer one (nodes: 256,128,4). In average, the three-layer requires around 1,300 – 1,700 epochs to obtain 13 scores, while the four-layer needs about 1,400 epochs. A straight forward tendency is to add more layers for a better performance in the future work.

Observation2. By changing the ReLU activation function into LeakyReLU, we can solve the environment in less than about 1000 epochs. This leaves a space for us to try other activation functions.

Observation3. Moreover, we find that it is relatively easy (e.g., a few hundred epochs from score 0. to 11.) for the agent to obtain an average score of around 11. However, it gets a bit harder (e.g., another few hundred epochs from score 11. to 13.) if we would like the agent to obtain an average score of 13. This indicates that new architectures that take into account the physical laws should be investigated.